114 - External login email confirmation

In this video we will discuss **how to confirm the email received from external login providers such as Google, Facebook etc**.

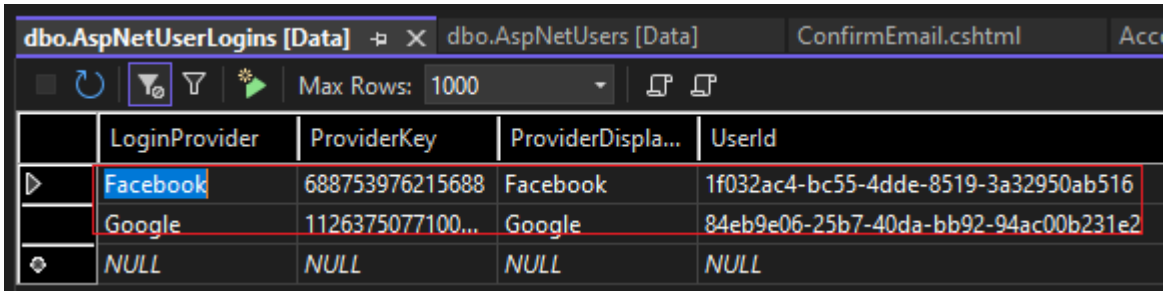
When we use external providers like Google or Facebook to login, we receive the user email from these external login provider. We then use this email to create a local user account.

Upon creating a local user account, send email confirmation link. If the email address is not confirmed, do not allow the user to login and display the error - Email not confirmed yet.

On the other hand, if the email address is confirmed, create an external login (AspNetUserLogins) and sign-in the user.

1) Create a new user
2) Generate token
3) Generate email confirmation link

The password hash is NULL for external login users.
Lat's delete our own external account from both tables: UserLogins + Users so we can star t from scratch:



dbo.AspNetUsers [Data] | ConfirmEmail.cshtml | AccountController.cs | Startup.cs

Max Rows: 1000

Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	PasswordHash
14fc38fd-8d41-...	Dima1@abv.bg	DIMA1@ABV.BG	Dima1@abv.bg	DIMA1@ABV.BG	False	AQAAAAEAAC...
1f032ac4-bc55-...	@protonmail.com	NIKOLETA.PAN...	nikoleta.panay...	NIKOLETA.PAN...	False	*NULL*
1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV....	aurora@abv.bg	AURORA@ABV....	False	AQAAAAEAAC...
3d9edac1-67d3...	Dima@abv.bg	DIMA@ABV.BG	Dima@abv.bg	DIMA@ABV.BG	False	AQAAAAEAAC...
8192bf2e-626b-...	Dima3@abv.bg	DIMA3@ABV.BG	Dima3@abv.bg	DIMA3@ABV.BG	False	AQAAAAEAAC...
84eb9e06-25b7...	amidalaflame@...	AMIDALAFLAM...	amidalaflame@...	AMIDALAFLAM...	False	*NULL*

ASP.NET Core Email Confirmation

Generate Email Confirmation Token

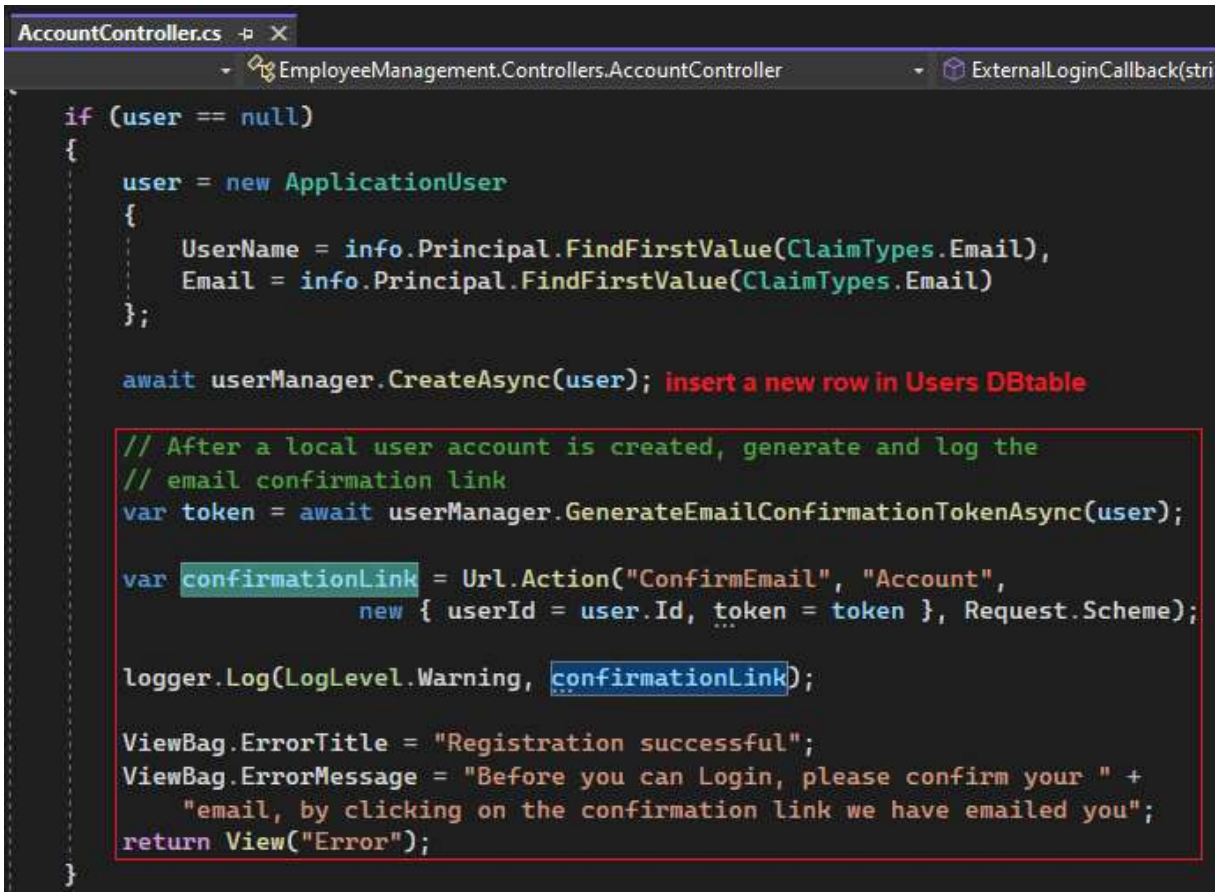
```
var token = await userManager.GenerateEmailConfirmationTokenAsync(user);
```

Generate Email Confirmation Link

```
var confirmationLink = Url.Action("ConfirmEmail", "Account",
    new { userId = user.Id, token = token }, Request.Scheme);
```

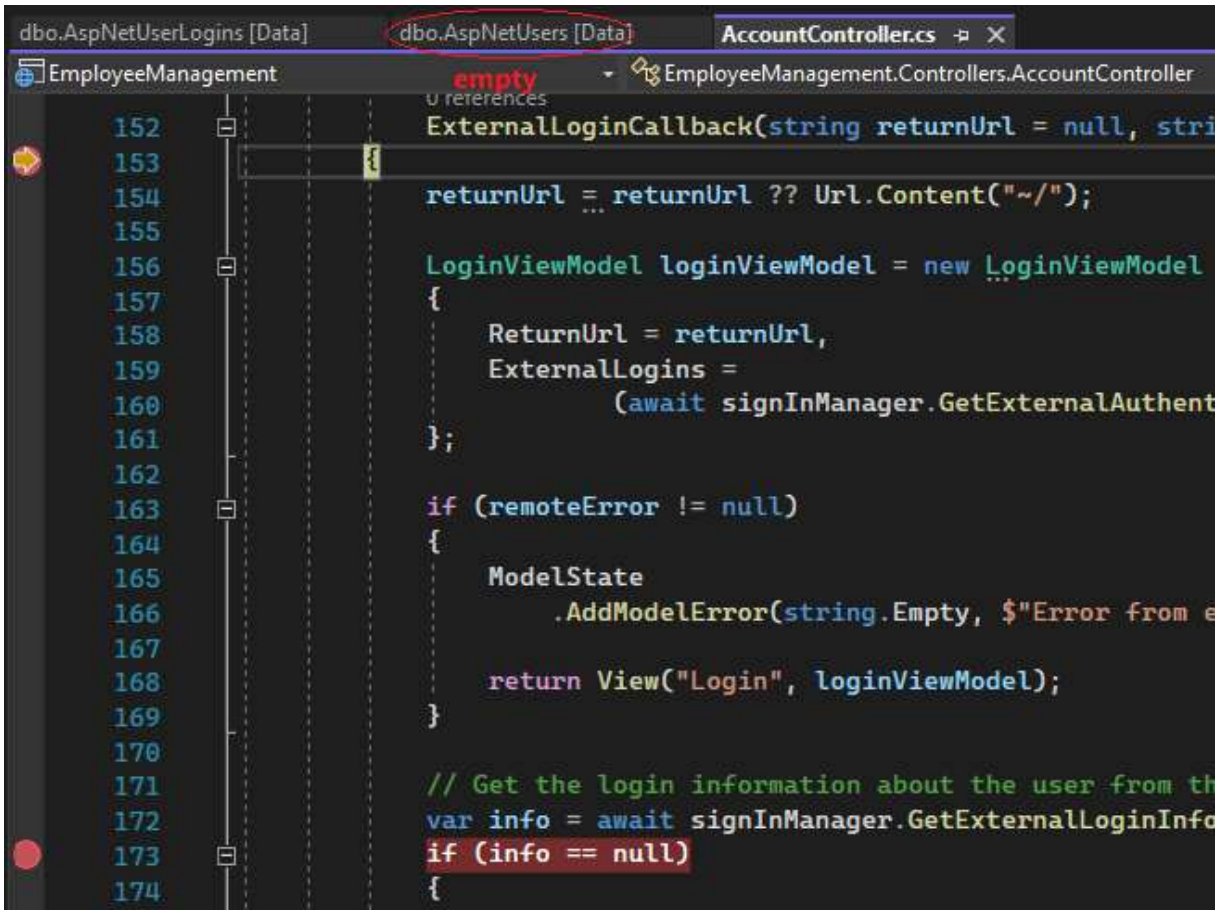
Generate Email Confirmation Link

```
logger.Log(LogLevel.Warning, confirmationLink);

ViewBag.ErrorTitle = "Registration successful";
ViewBag.ErrorMessage = "Before you can Login, please confirm your " +
    "email, by clicking on the confirmation link we have emailed you";
return View("Error");
```

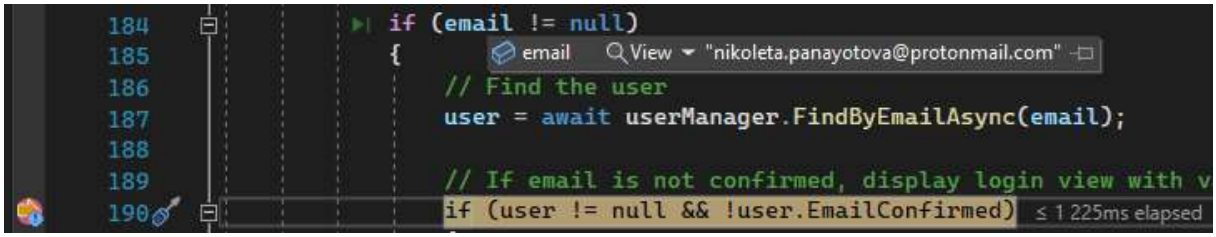


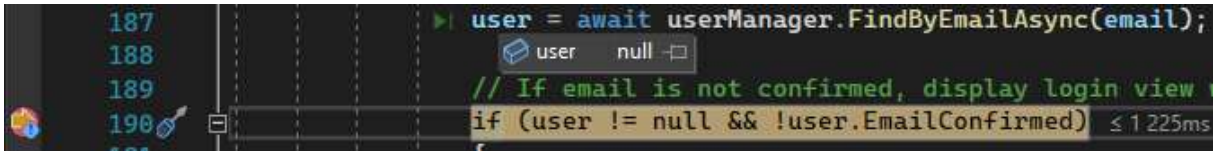

```
var token = await userManager.GenerateEmailConfirmationTokenAsync(user);

            var confirmationLink = Url.Action("ConfirmEmail", "Account",
                        new { userId = user.Id, token = token }, Request.Scheme);

            logger.Log(LogLevel.Warning, confirmationLink);

            ViewBag.ErrorTitle = "Registration successful";
            ViewBag.ErrorMessage = "Before you can Login, please confirm your " +
               "email, by clicking on the confirmation link we have emailed you";
            return View("Error");
```

Debug ->Facebook -
>ExternalCallBack action = BreakPoint1, Users table has no info (the user don't exist) -
> Continue = Breakpoint2



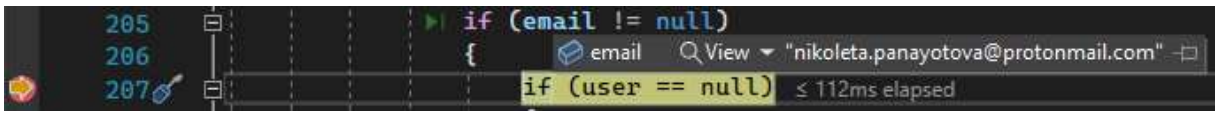
Continue->BP3



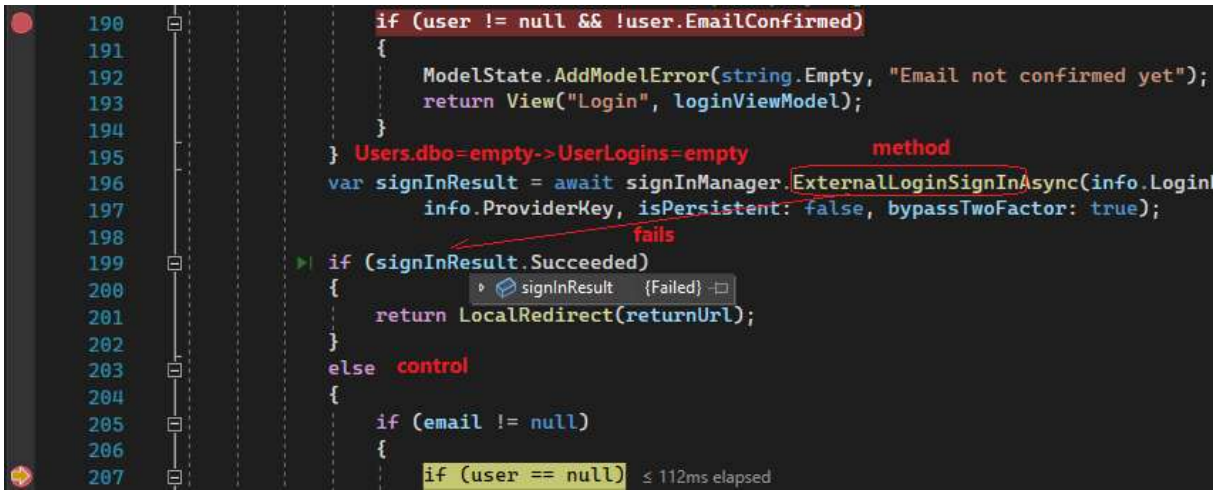


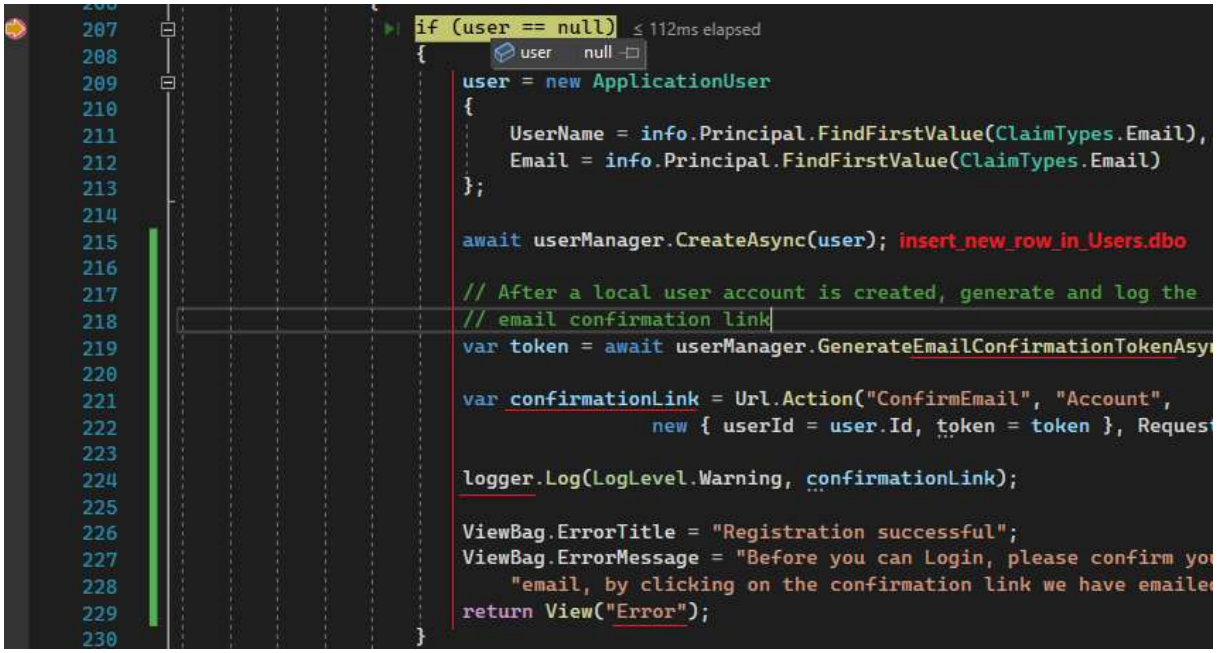
FindByEmail-
>null, there is no user with this email in Users.dbo, so this if {} will not be execited.

Continue->BP4

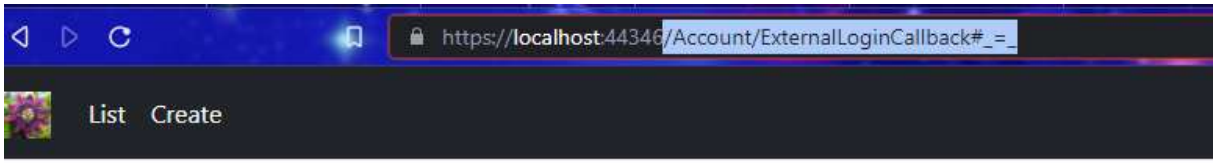


Users.dbo = empty -> UserLogins.dbo = empty





Continue->Message



Registration successful

Before you can Login, please confirm your email, by clicking on the confirmation link we have emailed you

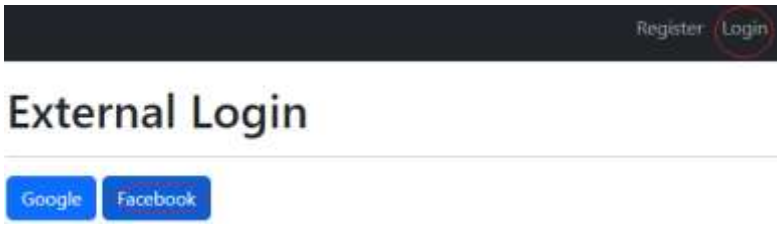
Users.dbo->Refresh->New user

dbo.AspNetUserLogins [Data] | dbo.AspNetUsers [Data] | AccountController.cs

Max Rows: 1000

Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed
14fc38fd-8d41-...	Dima1@abv.bg	DIMA1@ABV.BG	Dima1@abv.bg	DIMA1@ABV.BG	False
1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV....	aurora@abv.bg	AURORA@ABV....	False
3d9edac1-67d3...	Dima@abv.bg	DIMA@ABV.BG	Dima@abv.bg	DIMA@ABV.BG	False
8192bf2e-626b-...	Dima3@abv.bg	DIMA3@ABV.BG	Dima3@abv.bg	DIMA3@ABV.BG	False
8492084d-c18c-...	nikoleta.panay...	ROTONMAIL.COM	nikoleta.panay...	NIKOLETA.PAN...	False

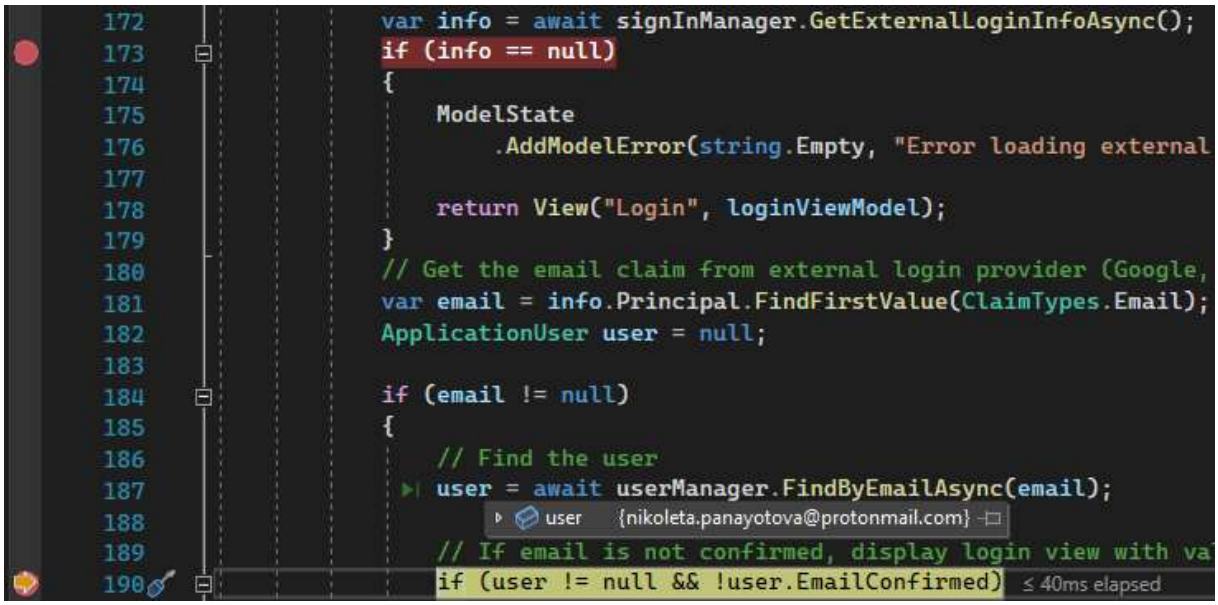
ConfirmEmails is False, se we can't login with it, but now the table is not null, user and email exists:



BP1->Continue->info is null -> Continue ->

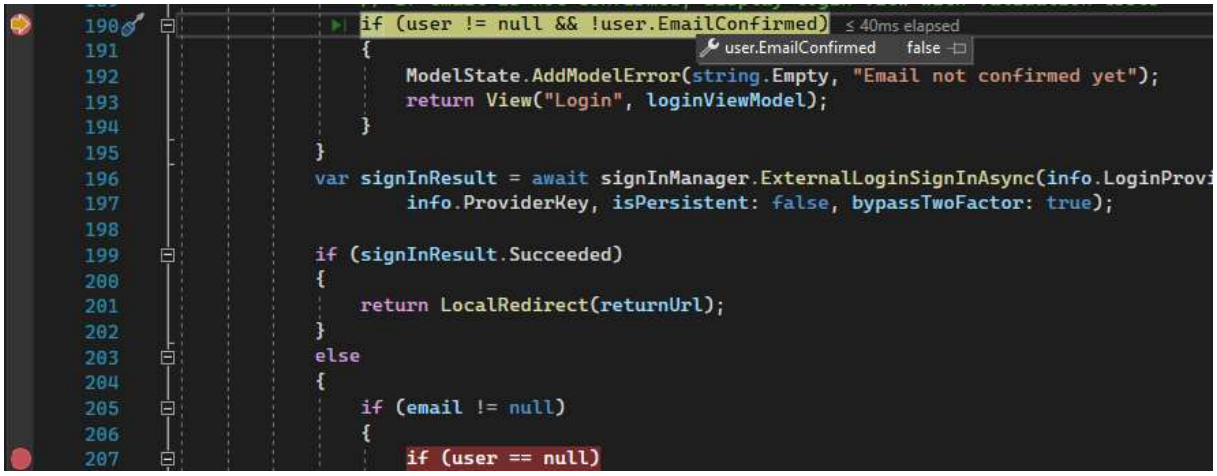
```
152     ExternalLoginCallback(string returnUrl = null, string remoteError = null)
153     {
154         returnUrl = returnUrl ?? Url.Content("~/");
155
156         LoginViewModel loginViewModel = new LoginViewModel
157         {
158             ReturnUrl = returnUrl,
159             ExternalLogins =
160                     (await signInManager.GetExternalAuthenticationSchemesAsync())
161         };
162
163         if (remoteError != null)
164         {
165             ModelState
166                 .AddModelError(string.Empty, $"Error from external provider: {rem
167
168             return View("Login", loginViewModel);
169         }
170
171         // Get the login information about the user from the external login provi
172         var info = await signInManager.GetExternalLoginInfoAsync();
173         if (inf  info  {Microsoft.AspNetCore.Identity.ExternalLoginInfo}
```

info has ExternalLoginInfo ->BP2



```
172             var info = await signInManager.GetExternalLoginInfoAsync();
173             if (info == null)
174             {
175                 ModelState
176                     .AddModelError(string.Empty, "Error loading external
177
178                 return View("Login", loginViewModel);
179             }
180             // Get the email claim from external login provider (Google,
181             var email = info.Principal.FindFirstValue(ClaimTypes.Email);
182             ApplicationUser user = null;
183
184             if (email != null)
185             {
186                 // Find the user
187                 user = await userManager.FindByEmailAsync(email);
188                                 user  {nikoleta.panayotova@protonmail.com}
189                 // If email is not confirmed, display login view with va
190                 if (user != null && !user.EmailConfirmed)  ≤ 40ms elapsed
```

Continue->User exists -> BP3



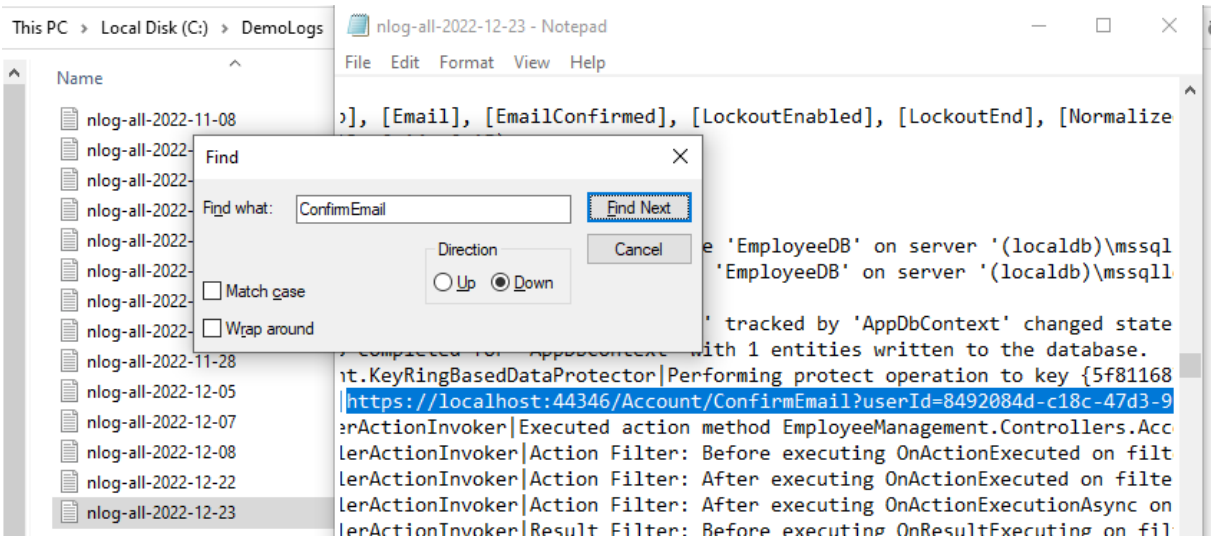
```
190             if (user != null && !user.EmailConfirmed)  ≤ 40ms elapsed
191             {                                  user.EmailConfirmed  false
192                 ModelState.AddModelError(string.Empty, "Email not confirmed yet");
193                 return View("Login", loginViewModel);
194             }
195         }
196         var signInResult = await signInManager.ExternalLoginSignInAsync(info.LoginProvi
197                 info.ProviderKey, isPersistent: false, bypassTwoFactor: true);
198
199         if (signInResult.Succeeded)
200         {
201             return LocalRedirect(returnUrl);
202         }
203         else
204         {
205             if (email != null)
206             {
207                 if (user == null)
```

email is not confirmed {false} -> Continue

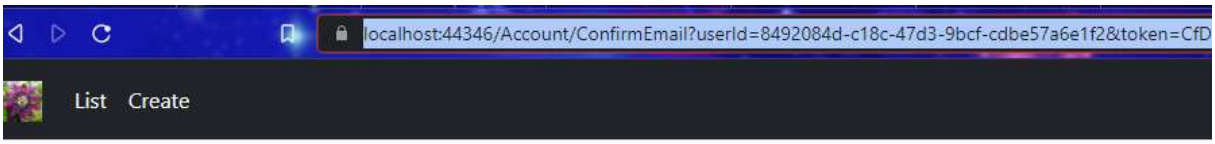
Local Account Login

- Email not confirmed yet

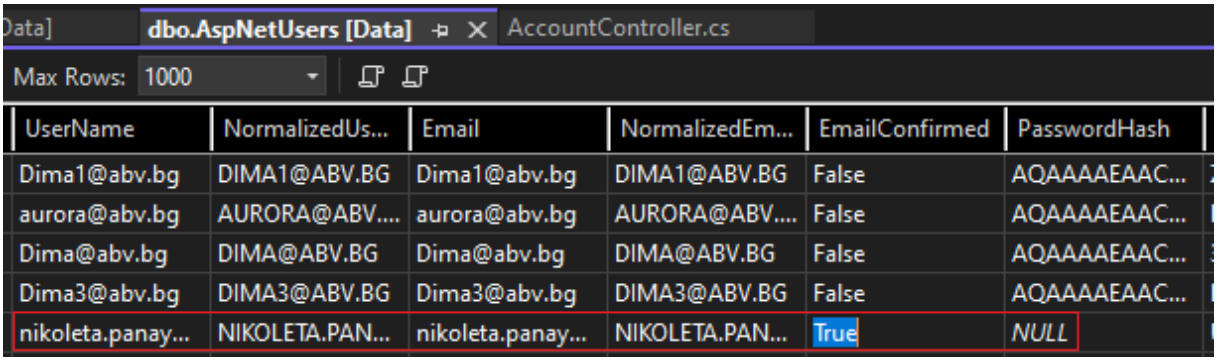
c:\DemoLogs\



https://localhost:44346/Account/ConfirmEmail?userId=8492084d-c18c-47d3-9bcf-cdbe57a6e1f2&token=CfDJ8IcWgV9y4PJKs6bemgk2FP4WyyeS2qx2651rbQBgdvqgs4Hk1VJ8tWW%2BqVxIvNMSpsJPqVX5I3qgqQ0qnS8CbJCqGHRr3I%2Bt8tqiwfUy%2BxwD6p9I9OSzFwvGfyTXbCrfZr2qyjJwlbRzhmXm0khLtwmrqu2Q2oV2%2FyfvTaccXZAQMvyZh%2BXTPg4gBFeJz0yYYNk3qnczcwJGXgQoLYmUlH1McszdbGBaZXw4TqkKvrDUgsTTi0SBtjq1E3plhvnkQA%3D%3D

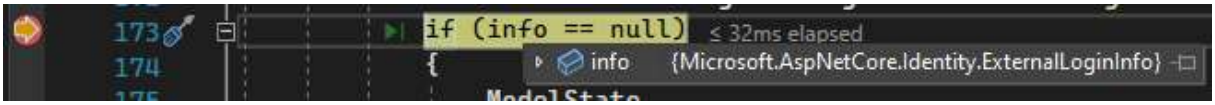


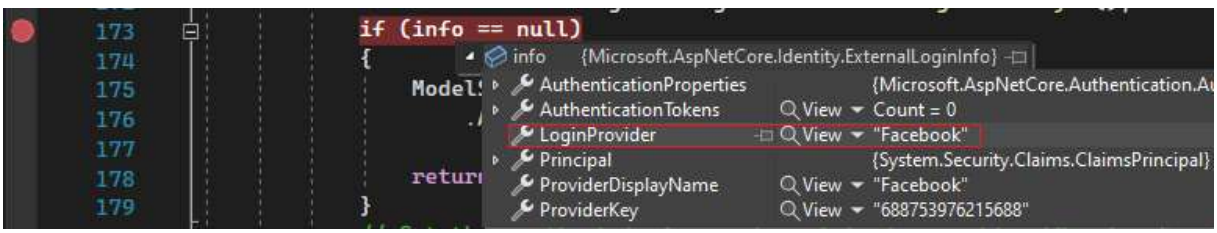
Thank you for confirming your email



UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	PasswordHash
Dima1@abv.bg	DIMA1@ABV.BG	Dima1@abv.bg	DIMA1@ABV.BG	False	AQAAAAEAAC...
aurora@abv.bg	AURORA@ABV....	aurora@abv.bg	AURORA@ABV....	False	AQAAAAEAAC...
Dima@abv.bg	DIMA@ABV.BG	Dima@abv.bg	DIMA@ABV.BG	False	AQAAAAEAAC...
Dima3@abv.bg	DIMA3@ABV.BG	Dima3@abv.bg	DIMA3@ABV.BG	False	AQAAAAEAAC...
nikoleta.panay...	NIKOLETA.PAN...	nikoleta.panay...	NIKOLETA.PAN...	True	NULL

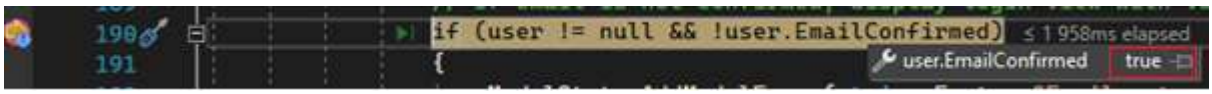
New test login in Debug mode from Facebook -> BP1->BP2->





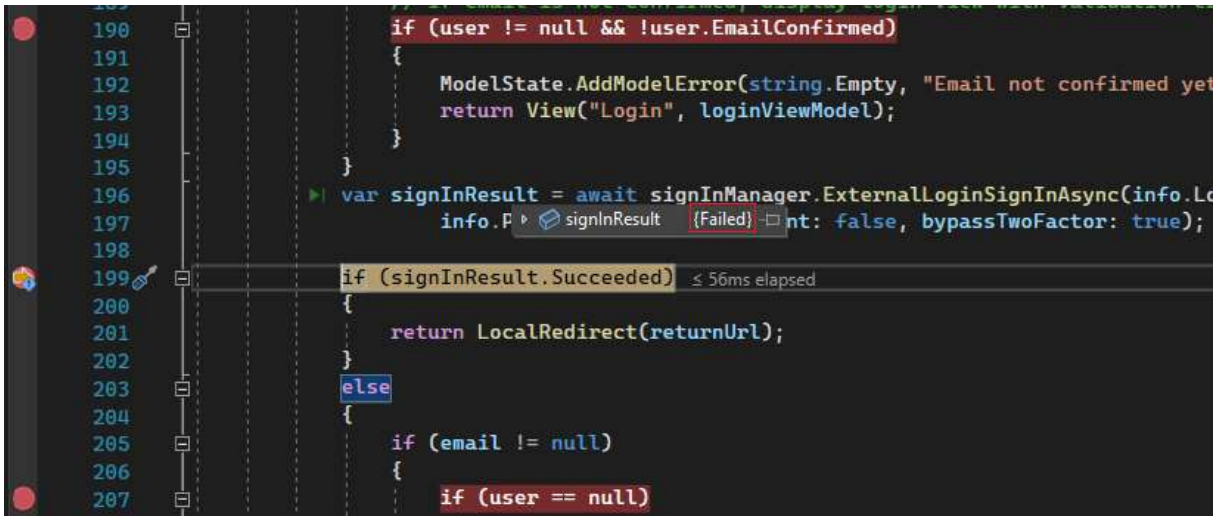
BP3

this time EmailConfirmed = True

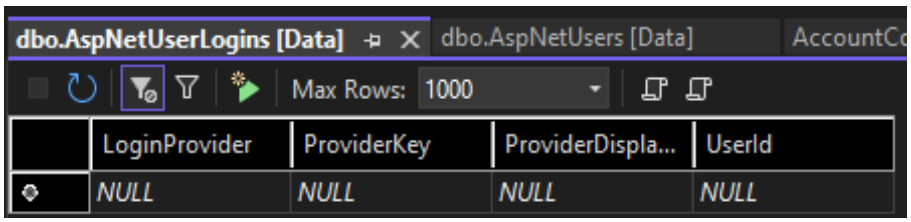


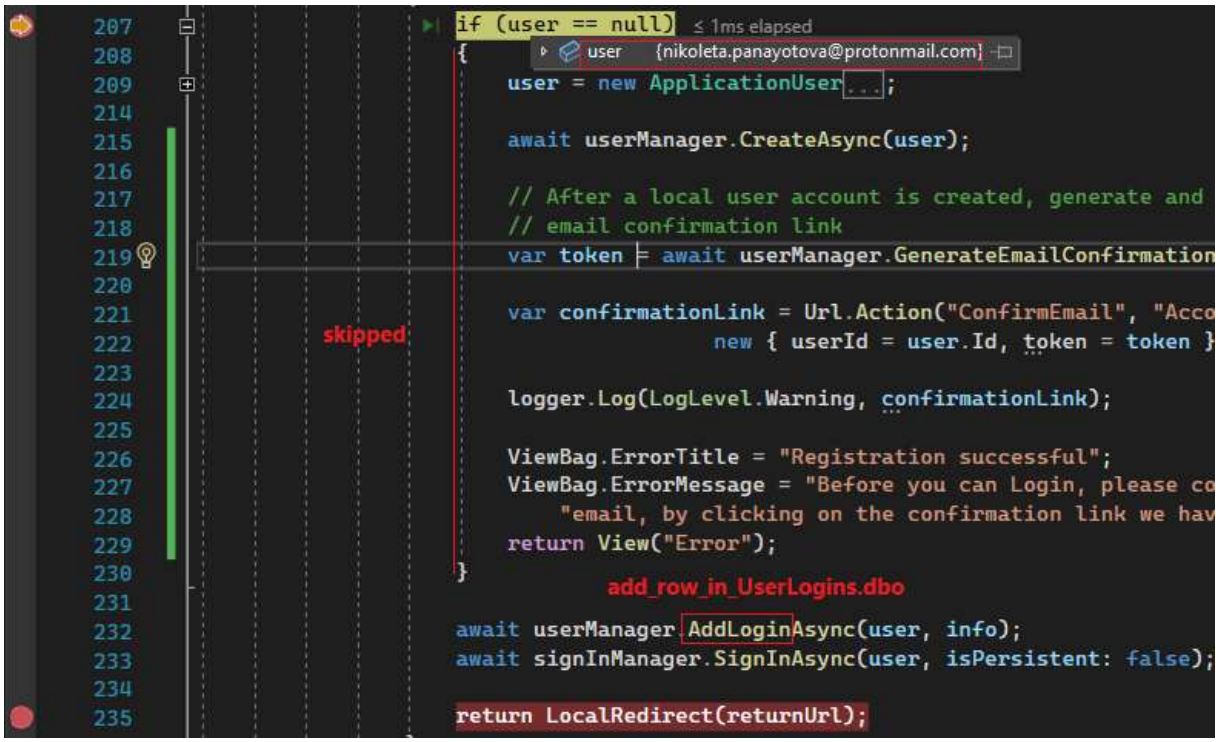
New BP5

`signInResult` = {Failed} for LoginProvider “Facebook”

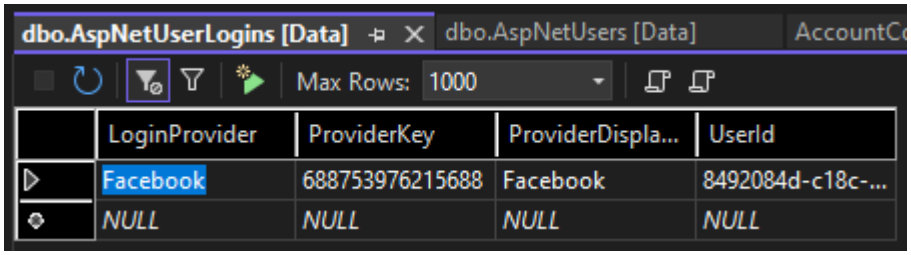


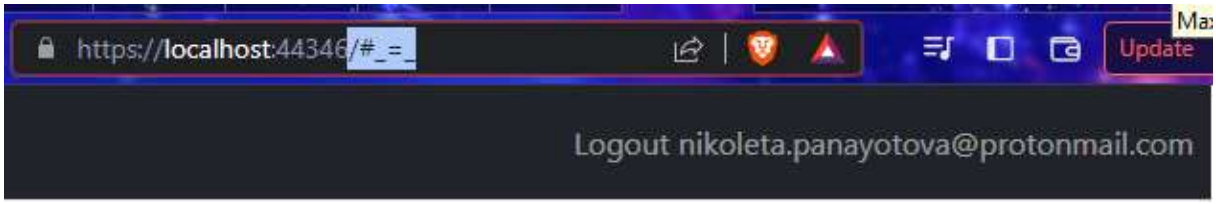
Because in UserLogins.dbo we don’t have user logins, so this method fails and we go to else block.





BP6

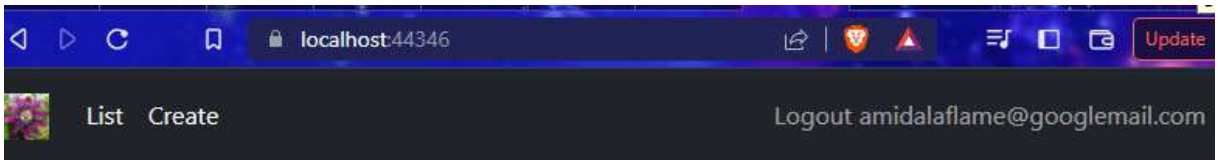




Logout->Remove all breakpoints->Login with the same email in Google account.

(I use different emails)

https://localhost:44346/Account/ConfirmEmail?userId=d5dcffab-a1ec-4aa6-aaf5-911d60422e4f&token=CfDJ8IcWgV9y4PJKs6bemgk2FP7SmQKGHH7mntemmhDtInLNzU8ziXuqKlGTGOhQZxZWqbSWhXxMdhjpsse6ERKLAOEzx8f5pdL34UTeSbbHWAvPnOwA6N%2BlcDf9M33yWqUKNggme84ScBL%2F7KaYMay51GC%2BNB9IDsQ3TtSLYghg%2FUs7YAYncGrasvZu%2BMh191yhe5LQuOssx0amaADD9h2Qw7Q%2BI45Mec0ujOqYnSpbYehK1xBS%2BWraCwW3j4XDDuw%2FFA%3D%3D



dbo.AspNetUserLogins [Data] | dbo.AspNetUsers [Data] | AccountC

Max Rows: 1000

	LoginProvider	ProviderKey	ProviderDispla...	UserId
	Facebook	688753976215688	Facebook	8492084d-c18c-...
▷	Google	1126375077100...	Google	d5dcffab-a1ec-...
	NULL	NULL	NULL	NULL

dbo.AspNetUserLogins [Data] | **dbo.AspNetUsers [Data]** | AccountController.cs

Max Rows: 1000

	Id	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed
	14fc38fd-8d41-...	Dima1@abv.bg	DIMA1@ABV.BG	Dima1@abv.bg	DIMA1@ABV.BG	False
	1f9ea9da-40d8-...	aurora@abv.bg	AURORA@ABV....	aurora@abv.bg	AURORA@ABV....	False
	3d9edac1-67d3...	Dima@abv.bg	DIMA@ABV.BG	Dima@abv.bg	DIMA@ABV.BG	False
	8192bf2e-626b-...	Dima3@abv.bg	DIMA3@ABV.BG	Dima3@abv.bg	DIMA3@ABV.BG	False
	8492084d-c18c-...	nikoleta.panay...	NIKOLETA.PAN...	nikoleta.panay...	NIKOLETA.PAN...	True
	87cabcf7-6319-...	sara@abv.bg	SARA@ABV.BG	sara@abv.bg	SARA@ABV.BG	False
	96944fa4-2400-...	Dima4@abv.bg	DIMA4@ABV.BG	Dima4@abv.bg	DIMA4@ABV.BG	False
	a5b9400f-45cf-...	george@abv.bg	GEORGE@ABV....	george@abv.bg	GEORGE@ABV....	False
	b14a1adf-9835-...	sofia@abv.bg	SOFIA@ABV.BG	sofia@abv.bg	SOFIA@ABV.BG	False
	b2056455-c2b4...	bob@abv.bg	BOB@ABV.BG	bob@abv.bg	BOB@ABV.BG	False
	c0ec608d-d07a...	Dima2@abv.bg	DIMA2@ABV.BG	Dima2@abv.bg	DIMA2@ABV.BG	False
	cf4120a9-5bca-...	david@abv.bg	DAVID@ABV.BG	david@abv.bg	DAVID@ABV.BG	False
	d2f456bd-46fc-...	tim@abv.bg	TIM@ABV.BG	tim@abv.bg	TIM@ABV.BG	False
▷	d5dcffab-a1ec-...	amidalaflame@...	AMIDALAFLAM...	amidalaflame@...	AMIDALAFLAM...	True
	f86605f9-0791-...	jason@abv.bg	JASON@ABV.BG	jason@abv.bg	JASON@ABV.BG	False
	NULL	NULL	NULL	NULL	NULL	NULL

```
[AllowAnonymous]
public async Task<IActionResult>
ExternalLoginCallback(string returnUrl = null, string remoteError = null)
{
    returnUrl = returnUrl ?? Url.Content("~/");

    LoginViewModel loginViewModel = new LoginViewModel
    {
        ReturnUrl = returnUrl,
        ExternalLogins =
        (await signInManager.GetExternalAuthenticationSchemesAsync()).ToList()
    };

    if (remoteError != null)
    {
        ModelState.AddModelError(string.Empty,
            $"Error from external provider: {remoteError}");
```

```
    return View("Login", loginViewModel);
}

var info = await signInManager.GetExternalLoginInfoAsync();
if (info == null)
{
    ModelState.AddModelError(string.Empty,
        "Error loading external login information.");

    return View("Login", loginViewModel);
}

var email = info.Principal.FindFirstValue(ClaimTypes.Email);
ApplicationUser user = null;

if (email != null)
{
    user = await userManager.FindByEmailAsync(email);

    if (user != null && !user.EmailConfirmed)
    {
        ModelState.AddModelError(string.Empty, "Email not confirmed yet");
        return View("Login", loginViewModel);
    }
}

var signInResult = await signInManager.ExternalLoginSignInAsync(
                        info.LoginProvider, info.ProviderKey,
                        isPersistent: false, bypassTwoFactor: true);

if (signInResult.Succeeded)
{
    return LocalRedirect(returnUrl);
}
else
{
    if (email != null)
    {
        if (user == null)
        {
            user = new ApplicationUser
            {
                UserName = info.Principal.FindFirstValue(ClaimTypes.Email),
                Email = info.Principal.FindFirstValue(ClaimTypes.Email)
            };

            await userManager.CreateAsync(user);

            // After a local user account is created, generate and log the
            // email confirmation link
            var token = await userManager.GenerateEmailConfirmationTokenAsync(user);

            var confirmationLink = Url.Action("ConfirmEmail", "Account",
                        new { userId = user.Id, token = token }, Request.Scheme);
```

```
                logger.Log(LogLevel.Warning, confirmationLink);

                ViewBag.ErrorTitle = "Registration successful";
                ViewBag.ErrorMessage = "Before you can Login, please confirm your " +
                    "email, by clicking on the confirmation link we have emailed you";
                return View("Error");
            }

            await userManager.AddLoginAsync(user, info);
            await signInManager.SignInAsync(user, isPersistent: false);

            return LocalRedirect(returnUrl);
        }

        ViewBag.ErrorTitle = $"Email claim not received from: {info.LoginProvider}";
        ViewBag.ErrorMessage = "Please contact support on Pragim@PragimTech.com";

        return View("Error");
    }
}
************************

[HttpGet]
[AllowAnonymous]
public async Task<IActionResult> ConfirmEmail(string userId, string token)
{
    if (userId == null || token == null)
    {
        return RedirectToAction("index", "home");
    }

    var user = await userManager.FindByIdAsync(userId);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"The User ID {userId} is invalid";
        return View("NotFound");
    }

    var result = await userManager.ConfirmEmailAsync(user, token);

    if (result.Succeeded)
    {
        return View();
    }

    ViewBag.ErrorTitle = "Email cannot be confirmed";
    return View("Error");
}
```